

Transformer Coding Interview Questions and Solutions

1. Implement scaled dot-product attention

Question: Implement `attention(Q, K, V, mask=None)`.

Tests: attention math, shapes, masking, softmax dimension.

```
import torch
import math

def scaled_dot_product_attention(Q, K, V, mask=None):
    """
    Q: [B, H, Tq, Dh]
    K: [B, H, Tk, Dh]
    V: [B, H, Tk, Dh]
    mask: broadcastable to [B, H, Tq, Tk]
    """
    scores = Q @ K.transpose(-2, -1)
    scores = scores / math.sqrt(Q.size(-1))

    if mask is not None:
        scores = scores.masked_fill(mask == 0, float("-inf"))

    weights = torch.softmax(scores, dim=-1)
    output = weights @ V

    return output, weights
```

2. Implement causal mask

Question: Create a mask so token `i` cannot attend to future tokens.

```
import torch

def causal_mask(seq_len, device=None):
    """Returns shape [1, 1, seq_len, seq_len]."""
    mask = torch.tril(torch.ones(seq_len, seq_len, device=device))
    return mask[None, None, :, :]
```

3. Implement multi-head self-attention

```
import torch
import torch.nn as nn
import math

class MultiHeadSelfAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0

        self.d_model = d_model
        self.num_heads = num_heads
        self.head_dim = d_model // num_heads
```

```

self.qkv_proj = nn.Linear(d_model, 3 * d_model)
self.out_proj = nn.Linear(d_model, d_model)

def forward(self, x, mask=None):
    B, T, D = x.shape

    qkv = self.qkv_proj(x)
    qkv = qkv.view(B, T, 3, self.num_heads, self.head_dim)
    qkv = qkv.permute(2, 0, 3, 1, 4)

    Q, K, V = qkv[0], qkv[1], qkv[2]

    scores = Q @ K.transpose(-2, -1)
    scores = scores / math.sqrt(self.head_dim)

    if mask is not None:
        scores = scores.masked_fill(mask == 0, float("-inf"))

    attn = torch.softmax(scores, dim=-1)
    out = attn @ V

    out = out.transpose(1, 2).contiguous()
    out = out.view(B, T, D)

    return self.out_proj(out)

```

4. Implement a Transformer feed-forward network

```

import torch.nn as nn

class FeedForward(nn.Module):
    def __init__(self, d_model, d_ff, dropout=0.1):
        super().__init__()

        self.net = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(d_ff, d_model),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)

```

5. Implement a decoder-only Transformer block

```

import torch.nn as nn

class TransformerBlock(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()

        self.ln1 = nn.LayerNorm(d_model)
        self.attn = MultiHeadSelfAttention(d_model, num_heads)
        self.ln2 = nn.LayerNorm(d_model)
        self.ffn = FeedForward(d_model, d_ff, dropout)

    def forward(self, x, mask=None):
        x = x + self.attn(self.ln1(x), mask)
        x = x + self.ffn(self.ln2(x))
        return x

```

6. Implement sinusoidal positional encoding

```

import torch
import math

def sinusoidal_positional_encoding(seq_len, d_model, device=None):
    pe = torch.zeros(seq_len, d_model, device=device)

    position = torch.arange(seq_len, device=device).unsqueeze(1)
    div_term = torch.exp(
        torch.arange(0, d_model, 2, device=device)
        * (-math.log(10000.0) / d_model)
    )

    pe[:, 0::2] = torch.sin(position * div_term)
    pe[:, 1::2] = torch.cos(position * div_term)

    return pe

```

7. Implement token + positional embeddings

```

import torch
import torch.nn as nn

class TokenPositionEmbedding(nn.Module):
    def __init__(self, vocab_size, max_seq_len, d_model):
        super().__init__()

        self.token_emb = nn.Embedding(vocab_size, d_model)
        self.pos_emb = nn.Embedding(max_seq_len, d_model)

    def forward(self, input_ids):
        B, T = input_ids.shape
        positions = torch.arange(T, device=input_ids.device)

        tok = self.token_emb(input_ids)
        pos = self.pos_emb(positions)[None, :, :]

        return tok + pos

```

8. Implement a tiny GPT model

```

import torch
import torch.nn as nn

class TinyGPT(nn.Module):
    def __init__(
        self,
        vocab_size,
        max_seq_len,
        d_model=128,
        num_heads=4,
        d_ff=512,
        num_layers=4,
        dropout=0.1,
    ):
        super().__init__()

        self.max_seq_len = max_seq_len
        self.embed = TokenPositionEmbedding(vocab_size, max_seq_len, d_model)

        self.blocks = nn.ModuleList([
            TransformerBlock(d_model, num_heads, d_ff, dropout)
            for _ in range(num_layers)
        ])

        self.ln_f = nn.LayerNorm(d_model)
        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)

```

```

def forward(self, input_ids):
    B, T = input_ids.shape

    if T > self.max_seq_len:
        raise ValueError("Sequence length exceeds max_seq_len")

    x = self.embed(input_ids)
    mask = causal_mask(T, device=input_ids.device)

    for block in self.blocks:
        x = block(x, mask)

    x = self.ln_f(x)
    logits = self.lm_head(x)

    return logits

```

9. Implement next-token prediction loss

```

import torch.nn.functional as F

def next_token_loss(logits, input_ids):
    """
    logits: [B, T, V]
    input_ids: [B, T]
    """
    shift_logits = logits[:, :-1, :].contiguous()
    shift_labels = input_ids[:, 1:].contiguous()

    loss = F.cross_entropy(
        shift_logits.view(-1, shift_logits.size(-1)),
        shift_labels.view(-1)
    )

    return loss

```

10. Implement greedy decoding

```

import torch

@torch.no_grad()
def generate_greedy(model, input_ids, max_new_tokens):
    model.eval()

    for _ in range(max_new_tokens):
        context = input_ids[:, -model.max_seq_len:]

        logits = model(context)
        next_logits = logits[:, -1, :]
        next_token = torch.argmax(next_logits, dim=-1, keepdim=True)

        input_ids = torch.cat([input_ids, next_token], dim=1)

    return input_ids

```

11. Implement temperature sampling

```

@torch.no_grad()
def generate_with_temperature(model, input_ids, max_new_tokens, temperature=1.0):
    model.eval()

    for _ in range(max_new_tokens):
        context = input_ids[:, -model.max_seq_len:]

```

```

logits = model(context)
logits = logits[:, -1, :] / temperature

probs = torch.softmax(logits, dim=-1)
next_token = torch.multinomial(probs, num_samples=1)

input_ids = torch.cat([input_ids, next_token], dim=1)

return input_ids

```

12. Implement top-k sampling

```

@torch.no_grad()
def generate_top_k(model, input_ids, max_new_tokens, k=50, temperature=1.0):
    model.eval()

    for _ in range(max_new_tokens):
        context = input_ids[:, -model.max_seq_len:]

        logits = model(context)
        logits = logits[:, -1, :] / temperature

        values, indices = torch.topk(logits, k)

        probs = torch.softmax(values, dim=-1)
        sampled = torch.multinomial(probs, num_samples=1)

        next_token = indices.gather(-1, sampled)

        input_ids = torch.cat([input_ids, next_token], dim=1)

    return input_ids

```

13. Implement top-p / nucleus sampling

```

@torch.no_grad()
def sample_top_p(logits, p=0.9):
    sorted_logits, sorted_indices = torch.sort(logits, descending=True)
    probs = torch.softmax(sorted_logits, dim=-1)
    cumulative_probs = torch.cumsum(probs, dim=-1)

    remove_mask = cumulative_probs > p
    remove_mask[:, 1:] = remove_mask[:, :-1].clone()
    remove_mask[:, 0] = False

    sorted_logits = sorted_logits.masked_fill(remove_mask, float("-inf"))

    filtered_probs = torch.softmax(sorted_logits, dim=-1)
    sampled = torch.multinomial(filtered_probs, num_samples=1)

    next_token = sorted_indices.gather(-1, sampled)

    return next_token

```

14. Implement padding mask for attention

```

def padding_mask(input_ids, pad_token_id):
    """
    input_ids: [B, T]
    returns: [B, 1, 1, T]
    """
    return (input_ids != pad_token_id)[:, None, None, :]

```

```

# Combine padding and causal masks:
# pad_mask = padding_mask(input_ids, pad_token_id)
# causal = causal_mask(T, input_ids.device)
# combined_mask = pad_mask & causal.bool()

```

15. Implement cross-attention

```

class CrossAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()

        assert d_model % num_heads == 0

        self.num_heads = num_heads
        self.head_dim = d_model // num_heads

        self.q_proj = nn.Linear(d_model, d_model)
        self.k_proj = nn.Linear(d_model, d_model)
        self.v_proj = nn.Linear(d_model, d_model)
        self.out_proj = nn.Linear(d_model, d_model)

    def forward(self, decoder_x, encoder_x, mask=None):
        B, T_dec, D = decoder_x.shape
        T_enc = encoder_x.size(1)

        Q = self.q_proj(decoder_x)
        K = self.k_proj(encoder_x)
        V = self.v_proj(encoder_x)

        Q = Q.view(B, T_dec, self.num_heads, self.head_dim).transpose(1, 2)
        K = K.view(B, T_enc, self.num_heads, self.head_dim).transpose(1, 2)
        V = V.view(B, T_enc, self.num_heads, self.head_dim).transpose(1, 2)

        scores = Q @ K.transpose(-2, -1)
        scores = scores / math.sqrt(self.head_dim)

        if mask is not None:
            scores = scores.masked_fill(mask == 0, float("-inf"))

        attn = torch.softmax(scores, dim=-1)
        out = attn @ V

        out = out.transpose(1, 2).contiguous().view(B, T_dec, D)

        return self.out_proj(out)

```

16. Implement RMSNorm

```

import torch
import torch.nn as nn

class RMSNorm(nn.Module):
    def __init__(self, d_model, eps=1e-8):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(d_model))

    def forward(self, x):
        rms = torch.sqrt(torch.mean(x ** 2, dim=-1, keepdim=True) + self.eps)
        x_norm = x / rms
        return self.weight * x_norm

```

17. Implement SwiGLU feed-forward layer

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class SwiGLUFFN(nn.Module):
    def __init__(self, d_model, d_hidden):
        super().__init__()

        self.w1 = nn.Linear(d_model, d_hidden)
        self.w2 = nn.Linear(d_model, d_hidden)
        self.w3 = nn.Linear(d_hidden, d_model)

    def forward(self, x):
        return self.w3(F.silu(self.w1(x)) * self.w2(x))

```

18. Count Transformer parameters

Approximate per block:

Attention QKV projections: $3 \times D \times D$
 Attention output projection: $D \times D$
 FFN: $D \times d_{\text{ff}} + d_{\text{ff}} \times D$
 Approximate block total: $4D^2 + 2D \times d_{\text{ff}}$

```

def estimate_params(vocab_size, d_model, d_ff, num_layers, tied_embeddings=True):
    block_params = 4 * d_model * d_model + 2 * d_model * d_ff
    total = num_layers * block_params

    total += vocab_size * d_model

    if not tied_embeddings:
        total += d_model * vocab_size

    return total

```

19. Implement embedding weight tying

```

class TinyGPTTied(nn.Module):
    def __init__(self, vocab_size, max_seq_len, d_model):
        super().__init__()

        self.token_emb = nn.Embedding(vocab_size, d_model)
        self.pos_emb = nn.Embedding(max_seq_len, d_model)
        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)

        self.lm_head.weight = self.token_emb.weight

```

20. Debug attention softmax dimension

Buggy code:

```

scores = Q @ K.transpose(-2, -1)
weights = torch.softmax(scores, dim=1)
out = weights @ V

```

Issue:

If scores shape is $[B, H, T_q, T_k]$, softmax should be over the key dimension, $\text{dim}=-1$, not over heads.

Correct:

```
scores = Q @ K.transpose(-2, -1)
scores = scores / math.sqrt(Q.size(-1))
weights = torch.softmax(scores, dim=-1)
out = weights @ V
```

21. Debug masking semantics

Buggy code:

```
scores = scores.masked_fill(mask == 1, float("-inf"))
```

If `mask == 1` means allowed, this masks the allowed positions.

Correct:

```
scores = scores.masked_fill(mask == 0, float("-inf"))
```

22. Implement gradient clipping

```
loss.backward()

torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

optimizer.step()
optimizer.zero_grad()
```

23. Implement learning-rate warmup with cosine decay

```
import math

def get_lr(step, max_lr, warmup_steps, total_steps, min_lr=0.0):
    if step < warmup_steps:
        return max_lr * step / warmup_steps

    progress = (step - warmup_steps) / (total_steps - warmup_steps)
    cosine = 0.5 * (1.0 + math.cos(math.pi * progress))

    return min_lr + cosine * (max_lr - min_lr)
```

24. Implement a simple training loop

```
def train_one_epoch(model, dataloader, optimizer, device):
    model.train()

    total_loss = 0.0

    for input_ids in dataloader:
        input_ids = input_ids.to(device)

        logits = model(input_ids)
        loss = next_token_loss(logits, input_ids)

        optimizer.zero_grad()
```



```

    loss.backward()

    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

    optimizer.step()

    total_loss += loss.item()

return total_loss / len(dataloader)

```

25. Implement KV cache conceptually

```

def attention_with_kv_cache(Q_new, K_new, V_new, cache=None):
    """
    Q_new: [B, H, 1, Dh]
    K_new: [B, H, 1, Dh]
    V_new: [B, H, 1, Dh]
    cache: dict with previous K and V
    """
    if cache is not None:
        K = torch.cat([cache["K"], K_new], dim=2)
        V = torch.cat([cache["V"], V_new], dim=2)
    else:
        K = K_new
        V = V_new

    new_cache = {"K": K, "V": V}

    scores = Q_new @ K.transpose(-2, -1)
    scores = scores / math.sqrt(Q_new.size(-1))

    weights = torch.softmax(scores, dim=-1)
    out = weights @ V

    return out, new_cache

```